```
Hello.propTypes = {
  name: PropTypes.string
}

document.addEventListener('DOMContentLoaded', () => {
  ReactDOM.render(
    <Hello name="React" />,
    document.body.appendChild(document.createElement('div')),
  )
})
```

We're going to use this app to verify Webpacker is set up correctly. First, we need to generate a page that the React app will be loaded on:

```
$ docker-compose exec web bin/rails g controller pages home
      create  app/controllers/pages_controller.rb
       route  get 'pages/home'
      invoke  erb
      create    app/views/pages
      create    app/views/pages/home.html.erb
      invoke  helper
      create    app/helpers/pages_helper.rb
      invoke  assets
      invoke    coffee
      create      app/assets/javascripts/pages.coffee
      invoke    scss
      create      app/assets/stylesheets/pages.scss
```

Let's modify the generated view (app/views/pages/home.html.erb) to load the React app; while we are at it, let's delete the default content and give the page a new title:

```
<%= javascript_pack_tag 'hello_react' %>

<h1>React App</h1>
```

OK, let's try this out. Navigate to http://localhost:3000/pages/home, and you should see the content "Hello React!" displayed on the page. This confirms that our React app is being compiled and loaded correctly.

Auto-updating works too. Update app/javascript/packs/hello_react.jsx to set the defaultProps.name to your name:

```
<Hello name="<Your name>" />
```

Now when you reload the browser, you should see the page has updated (unless your name, coincidentally, is "React").

Thrilling, this app isn't. But with these basics in place, you can now develop with React as part of your Rails app, and build whatever you like.